

Відокремлений структурний підрозділ
«Полтавський політехнічний фаховий коледж
Національного технічного університету
«Харківський політехнічний інститут»

Циклова комісія дисциплін програмної інженерії

МЕТОДИЧНА РОЗРОБКА ПОКАЗОВОГО ЗАНЯТТЯ

з дисципліни «Програмування»

на тему: **Робота з графічними бібліотеками**

ПОЛТАВА 2025

Автор: Владислав ОЛІЙНИК

Викладач дисциплін інженерії програмного забезпечення, спеціаліст другої кваліфікаційної категорії

Методична розробка розглянута і схвалена на засіданні ЦК дисциплін програмної інженерії

Протокол №3 від «24» жовтня 2025 року

Голова ЦК _____ Олександр БАБИЧ

Тема заняття: Робота з графічними бібліотеками

Мета заняття:

навчальна: Сформувати у студентів фундаментальне розуміння архітектури SFML як інструменту для розробки 2D-ігор та додатків, а також розвинути практичні навички, необхідні для реалізації основного життєвого циклу графічної програми: від ініціалізації вікна та завантаження зовнішніх ресурсів до відображення об'єктів та обробки користувацького вводу в межах постійного ігрового циклу;

розвивальна: Розвинути системне та алгоритмічне мислення студентів через засвоєння концепції "Ігрового Циклу" та обробки подій, а також сформувати навички структурування коду та ефективного використання об'єктно-орієнтованих підходів для вирішення графічних задач (як-от управління ресурсами та анімація);

виховна: Сприяти формуванню культури програмування, самостійності та наполегливості у вирішенні технічних завдань, а також виховувати повагу до авторських прав при роботі з графічними ресурсами.

Тип заняття: засвоєння нових знань

Форма заняття: лекційне заняття

Обладнання і матеріали:

Викладач: мультимедійний проектор, презентація (доступна за посиланням <https://tinyurl.com/y8teu2fb>);

Здобувачі освіти: мобільний телефон , зошит, ручка.

Основні поняття: поняття графічної бібліотеки, основні модулі SFML (системний, віконний, графічний, аудіо, мережевий), огляд програмного забезпечення спеціалізованого призначення.

План лекції:

1. Огляд та налаштування бібліотеки SFML
2. Виклад основного(теоретичного) матеріалу лекції
3. Практичне використання бібліотеки SFML(показ проектів в реальному часі)

Хід заняття:

I. Організаційний момент.

II. Оголошення теми та мети заняття. Мотивація навчальної діяльності.

1. Повідомлення теми заняття

- На занятті ми будемо розглядати сторонню графічну бібліотеку SFML в межах дисципліни програмування. (Слайд 1)
- Оголосити мету заняття: Надати студентам базові теоретичні знання про бібліотеку SFML (Simple and Fast Multimedia Library), її архітектуру, основні модулі та принципи використання для створення графічних застосунків та 2D-ігор мовою C++. (Слайд 2)
- Наголосити на основних завданнях заняття. (Слайд 3 та Слайд 4)

2. Огляд та налаштування бібліотеки SFML

Завантаження бібліотеки SFML

- Проскануйте цей QR-код, та перейдіть за посиланням, з метою завантаження графічної бібліотеки. (Слайд 5)
- Далі використовуючи покрокову інструкцію з помітками завантажити графічну бібліотеку, котра відповідає вашій операційній системі, з подальшим розархівуванням бібліотеки на жорсткому диску.(Слайди 6-9)

Налаштування бібліотеки SFML

- Перейдіть в середовище розробки та створіть пустий проект мовою C++. Використовуючи покрокову інструкцію з помітками виконайте всі кроки налаштування бібліотеки. (Слайди 10-22)
- Тепер Ви готові розпочинати вивчати дану графічну бібліотеку!

3. Виклад основного матеріалу лекції

Оголошення теми

- Сьогодні ми розпочинаємо, мабуть, одну з найцікавіших і найвізуальніших тем у нашому курсі — роботу з графічною бібліотекою SFML. Якщо ви коли-небудь мріяли створити власну гру чи просто зрозуміти, як комп'ютер малює об'єкти на екрані, це заняття — ваш перший крок до цієї мети. (Слайд 23)

III. Робота з новим матеріалом.

Виклад теоретичного матеріалу

- Ми вже знаємо, що SFML — це не монолітний рушій, а набір інструментів. Він поділений на п'ять незалежних модулів, які дозволяють нам організовувати код і підключати лише ті функції, які дійсно потрібні. Це робить наші програми швидкими та гнучкими. Сьогодні ми розглянемо ці п'ять стовпів SFML. Кожен модуль відповідає за свою окрему, критично важливу сферу мультимедійної програми. (Слайд 24)
- Давайте спробуємо розібратись з кожним модулем окремо. Розпочнемо з системного модулю. (Слайд 25)

Це фундамент усієї бібліотеки. System містить базові класи, які використовуються іншими модулями, але самі не мають прямого відношення до графіки чи звуку.

- ✓ Ключова роль полягає в наданні класів для роботи з часом (`sf::Clock`, `sf::Time`), керування потоками, а також деякі векторні та невеликі математичні структури.
 - ✓ Чому важливий? Клас `sf::Time` є критично важливим для створення плавної анімації, незалежної від швидкості процесора (концепція *delta time*)."
- Далі перейдемо до розгляду віконного модулю. (Слайд 26)

Цей модуль — ваш прямий інтерфейс з операційною системою.

- ✓ Він дозволяє створити вікно (`sf::RenderWindow`), встановити його розміри, заголовок, керувати режимами відображення та, найголовніше, обробляти події (`sf::Event`) від користувача — натискання клавіш, рухи миші, закриття вікна.
 - ✓ Без нього не існує 'полотна' для малювання.
 - ✓ Обробляє взаємодію з користувачем (введення з клавіатури, миші, джойстика).
 - ✓ Надає контекст для OpenGL-рендерингу (якщо ви хочете використовувати 3D або більш складну 2D графіку).
- Далі будемо розглядати графічний модуль. (Слайд 27)

Модуль `sf::Graphics` відповідає за все, що ви бачите на екрані. Його головна місія — забезпечити ефективний зв'язок між вашою програмною логікою (обчисленнями координат) та апаратним забезпеченням комп'ютера (відеокартою), щоб відобразити кінцевий результат.

- ✓ Прості Форми (`sf::Shape`). Це будівельні блоки для початківців. Класи, як-от `sf::CircleShape` або `sf::RectangleShape`, дозволяють швидко малювати геометричні примітиви, встановлювати їхній колір, розмір і позицію. Вони ідеальні для тестування та створення простого інтерфейсу.
- ✓ `sf::Texture` (Текстура). Це сирі дані зображення, завантажені з файлу (наприклад, `.png` або `.jpg`) і збережені у пам'яті відеокарти. Текстура не має позиції чи розміру на екрані.
- ✓ `sf::Sprite` (Спрайт). Це об'єкт, який відображає частину або всю текстуру. Він має позицію, може бути масштабований, повернений і є тим, що ми фактично малюємо на екрані. Це класичний ООП-приклад: текстура — це дані, спрайт — це відображення даних.
- ✓ Візуалізація Тексту. Клас `sf::Text` дозволяє відображати текстову інформацію (наприклад, рахунок гравця або меню), використовуючи завантажені шрифти (`sf::Font`).

- Переходимо до розгляду Аудіо модуля. (Слайд 28)

Модуль `sf::Audio` відповідає за всю звукову взаємодію у вашій програмі. Його основна мета — забезпечити простий та ефективний спосіб завантаження, відтворення та керування звуковими файлами.

Хоча SFML є 2D-бібліотекою, модуль `Audio` підтримує концепцію позиціонування звуку. Це означає, що ми можемо визначити положення джерела звуку (наприклад, вибуху) у просторі. Якщо ваш гравець знаходиться близько до нього, звук буде гучним, якщо далеко — тихим. Це додає іграм глибини та реалізму.

- ✓ `sf::SoundBuffer` — це контейнер, який завантажує звуковий файл (наприклад, постріл, клацання, стрибок) у пам'ять один раз. Це ідеально для коротких, часто повторюваних звуків, оскільки вони відтворюються миттєво.
- ✓ `sf::Sound` — це відтворюваний екземпляр, який використовує дані з буфера. Ви можете мати багато об'єктів `sf::Sound`, що використовують один і той самий буфер.

- Розглянемо останній з модулів, а саме мережевий. (Слайд 29)

Модуль `sf::Network` надає базові, але потужні інструменти для мережевої комунікації, абстрагуючи вас від складнощів системного програмування сокетів. Він дозволяє вашій програмі стати частиною глобальної мережі.

Його основна увага зосереджена на двох ключових сферах:

- ✓ TCP (Transmission Control Protocol). Надає надійне, орієнтоване на підключення з'єднання. Це схоже на телефонний дзвінок: дані гарантовано прийдуть у правильному порядку, але це вимагає більше часу на встановлення з'єднання. Ідеально для передачі важливих даних (наприклад, реєстраційних даних, логіки гри).
 - ✓ UDP (User Datagram Protocol). Надає швидке, але ненадійне з'єднання. Це схоже на листівку: дані можуть прийти не в порядку або загубитися, але швидкість відправки дуже висока. Ідеально для даних, які постійно оновлюються (наприклад, позиція гравця в реальному часі).
 - ✓ HTTP-запитів (sf::Http). Для взаємодії з вебсерверами (наприклад, отримання таблиці лідерів з онлайн-сервісу).
 - ✓ Пакетів (sf::Packet). Це зручний контейнер для безпечного упакування та розпакування різних типів даних (числа, рядки) перед їх надсиланням через мережу.
- Ми розглянули, з яких модулів складається SFML, і що він дозволяє робити. Перш ніж глибоко занурюватися у кодування, важливо чітко розуміти: чи є SFML найкращим інструментом саме для наших завдань?

Як і будь-яка технологія, SFML має свої сильні та слабкі сторони. Розуміння цих аспектів допоможе нам приймати обґрунтовані рішення щодо архітектури наших майбутніх проєктів.

Давайте коротко розглянемо основні переваги (плюси), які зробили SFML популярним, та недоліки (мінуси), які ми повинні враховувати в роботі. (Слайд 30)

Переваги (Сильні Сторони SFML):

- ✓ Простота та Зрозумілість (Easy to Learn). SFML має набагато менший поріг входження, ніж DirectX чи OpenGL. Його об'єктно-орієнтований API на C++ є чистим і логічним, що дозволяє швидко розпочати малювання та роботу зі звуком.
- ✓ Кросплатформеність SFML підтримує роботу на Windows, Linux, macOS та навіть деяких мобільних платформах. Написаний вами код, як правило, легко компілюється та запускається на різних операційних системах.

- ✓ Висока Продуктивність (Fast). Будучи бібліотекою, написаною на C++ і мінімально абстрагуючою від заліза, SFML забезпечує відмінну швидкість роботи, особливо для 2D-графіки.
- ✓ Повний Контроль. На відміну від 'великих' ігрових рушіїв, які часто приховують логіку, SFML надає вам повний контроль над тим, як і коли виконується кожен рядок коду, що є безцінним для навчання.
- Ми вже знаємо, що модуль `sf::Window` відповідає за створення інтерфейсу вашої програми. У цьому модулі є один клас, який є абсолютно центральним для будь-якого графічного проекту: `sf::RenderWindow`. По суті, `sf::RenderWindow` виконує дві критично важливі функції, об'єднуючи функціонал двох модулів:
 - ✓ Вікно (Window). Він створює та керує видимим вікном на екрані. Це 'рамка', яка взаємодіє з операційною системою, дозволяючи вам його переміщувати, змінювати розмір, згортати та закривати.
 - ✓ Ціль Рендерингу (Render Target). Це 'полотно' для малювання. Усі графічні об'єкти (форми, спрайти, текст) можуть бути намальовані лише на об'єкті, який є ціллю рендерингу. Наш `sf::RenderWindow` є основною ціллю. (Слайд 31)
- Ми вже знаємо, що клас `sf::RenderWindow` є нашим командним центром. Щоб створити об'єкт цього класу, ми викликаємо його конструктор, якому обов'язково потрібно передати два основні аргументи. Ці аргументи визначають, як саме виглядатиме та функціонуватиме наше вікно з моменту його появи. (Слайд 32)
- Перший аргумент використовує клас `sf::VideoMode` і відповідає за фізичні параметри вікна:
 - ✓ Розміри (Width та Height). У середині `sf::VideoMode` ми вказуємо ширину та висоту вікна у пікселях. Наприклад, `sf::VideoMode(800, 600)` створить вікно з розмірами 800 пікселів у ширину та 600 у висоту.
 - ✓ Глибина Кольору (Optional). Третім, необов'язковим, параметром можна вказати глибину кольору (наприклад, 32 біти), але для більшості 2D-ігор це можна пропустити, використовуючи стандартні налаштування. (Слайд 33)
- У SFML ви часто зустрінете тип даних `sf::Uint32`. Хоча він не є прямим аргументом конструктора `sf::RenderWindow`, він є фундаментальним для роботи з графікою, особливо при визначенні кольорів та деяких параметрів

`sf::VideoMode`. Призначення у SFML: У SFML `sf::Uint32` використовується для двох основних цілей:

- ✓ Колір (RGBA). Найчастіше `sf::Uint32` використовується для зберігання повного 32-бітного коду кольору RGBA (Червоний, Зелений, Синій, Альфа-канал). Кожен із чотирьох каналів (R, G, B, A) займає 8 біт (або 1 байт), що в сумі дає 32 біти.
 - ✓ `Style::None` - Вікно без рамки.
 - ✓ `Style::Titlebar` - Вікно тільки з заголовком.
 - ✓ `Style::Resize` - Дозволяє користувачеві змінювати розмір.
 - ✓ `Style::Close` - Має кнопку закриття ('X').
 - ✓ `Style::Fullscreen` - Вікно на весь екран. (Слайд 34)
- Подія — це будь-яка значуща дія, яка походить від користувача чи системи і яку ми хочемо відстежувати. SFML фіксує ці дії та складає їх у чергу.

`sf::Event` — це структура-контейнер, яка може зберігати інформацію про будь-яку з можливих подій. Це може бути:

- ✓ Системні події. Вікно закрито, змінено розмір, фокус на вікні втрачено.
- ✓ Введення з Клавіатури. Клавiша натиснута (`KeyPressed`), клавiша відпущена (`KeyReleased`).
- ✓ Введення з Миші. Кнопка миші натиснута, миша переміщена (`MouseMoved`), прокручування коліщатка.

Обробка подій відбувається всередині нашого Ігрового Циклу і складається з трьох ключових кроків:

- ✓ Цикл Опитування (`while (window.pollEvent(event))`). На початку кожного кадру ми опитуємо вікно. Метод `pollEvent()` бере першу подію з черги, поміщає її в нашу змінну `event` і повертає `true`. Якщо черга порожня, він повертає `false`. Це дозволяє нам обробити всі події, що відбулися з моменту останнього кадру.
- ✓ Визначення Типу Події (`event.type`). Оскільки структура `event` може містити будь-яку подію, ми використовуємо оператор `switch` (або `if/else`) для визначення її типу (`event.type`).
- ✓ Реагування. Залежно від типу, ми виконуємо відповідну дію. Наприклад, якщо `event.type == sf::Event::Closed`, ми викликаємо `window.close()`. (Слайд 35)

- Рендеринг (від англ. render — візуалізувати, відтворювати) — це процес перетворення даних (координат, текстур, кольорів) у готову картинку, яка відображається на моніторі.

У SFML, як і в більшості сучасних графічних програм, рендеринг відбувається постійно, кадр за кадром, усередині нашого Ігрового Циклу (Game Loop). Щоб уникнути мерехтіння і забезпечити плавне оновлення, SFML використовує техніку подвійної буферизації. Кожен кадр, який ви бачите, створюється за допомогою чіткої послідовності з трьох команд:

- ✓ `window.clear(color)` - Очищає вікно. Зафарбовує його в певний колір. Має бути викликана першою у фазі рендерингу.
 - ✓ `window.draw(drawable)` - Малює об'єкт, який реалізує інтерфейс.
 - ✓ `window.display()` - Відображає на екрані все, що було намальовано, виконується останньою! (Слайд 36)
- Ми навчилися малювати об'єкти на екрані, але досі ми працювали з припущенням, що наша камера (те, що бачить користувач) завжди має той самий розмір і позицію, що й вікно.

Насправді, у SFML є потужний механізм, який дозволяє нам керувати тим, що саме користувач бачить, незалежно від фізичного розміру вікна. Цей механізм складається з двох ключових концепцій: Viewport (Область перегляду) та View (Вигляд або Камера). View та Viewport працюють у парі, створюючи потужний механізм з такими функціями.

- ✓ `window.setView(sf::View)` - Встановлює поточний Вид, який визначає, яка частина ігрового світу буде відображатися у вікні.
 - ✓ `window.getSize()` - Повертає розмір вікна
 - ✓ `window.setFramerateLimit(limit)` - Обмежує кількість кадрів на секунду (FPS), щоб зменшити навантаження на CPU/GPU. (Слайд 37)
- У SFML ми не малюємо пікселі вручну. Натомість, ми використовуємо об'єктно-орієнтований підхід: ми створюємо об'єкти-фігури і потім вказуємо вікну, де і як їх намалювати.

SFML надає кілька вбудованих класів, які успадковують функціонал від базового `sf::Shape`. Сьогодні ми зосередимося на двох найпоширеніших:

- ✓ `sf::RectangleShape` (Прямокутник). Ідеально підходить для платформ, стін, базових елементів інтерфейсу (GUI) та більшості ігрових об'єктів.
- ✓ `sf::CircleShape` (Коло): Використовується для м'ячів, планет, аватарів або будь-чого, що вимагає ідеально круглої форми. (Слайд 38)

- Ми вже навчилися створювати об'єкти фігур, такі як `sf::RectangleShape` та `sf::CircleShape`. Проте, для того, щоб ці фігури стали повноцінними елементами нашої сцени, їх потрібно налаштувати.

Усі класи, що успадковують від `sf::Shape`, мають спільний набір властивостей (атрибутів), які ми можемо змінювати за допомогою спеціальних методів. Ці властивості визначають, як фігура виглядає, де вона розташована та як вона взаємодіє з іншими елементами.

- ✓ `setFillColor(color)` - Встановлює колір заливки фігури.
- ✓ `setOutlineColor(color)` - Встановлює колір контуру (обведення).
- ✓ `setOutlineThickness(thickness)` - Встановлює товщину контуру (позитивне значення — назовні, негативне — всередину).
- ✓ `setPosition(x, y)` - Встановлює позицію фігури у вікні.
- ✓ `setRotation(angle)` - Встановлює кут повороту фігури (у градусах).
- ✓ `setScale(factorX, factorY)` - Встановлює масштаб фігури.
- ✓ `move(offsetX, offsetY)` - Зміщує фігуру на вказану відстань (використовується для анімації). (Слайд 39 та Слайд 40)

- Клас `Texture` (Текстура) є ключовим елементом у модулі `Graphics SFML` і відповідає за завантаження та зберігання зображень у відеопам'яті (GPU). Це необхідно для ефективного відображення статичних зображень, таких як спрайти, елементи інтерфейсу та фони. Основною функцією якої є ефективне зберігання. Зберігання даних зображення безпосередньо у VRAM (відеопам'яті). Це дозволяє графічному процесору (GPU) швидко отримувати доступ до даних під час рендерингу, що критично важливо для високої продуктивності. Для роботи з текстурами використовують основні функції:

- ✓ `loadFromFile(const std::string& filename)` - Основний спосіб. Завантажує зображення з файлу (підтримує більшість поширених форматів: PNG, JPG, GIF, BMP).
- ✓ `loadFromMemory(const void* data, std::size_t size)` - Завантажує зображення з даних, що зберігаються в оперативній пам'яті (наприклад, з масиву байтів).
- ✓ `loadFromImage(const sf::Image& image)` - Завантажує текстуру з об'єкта
- ✓ `create(unsigned int width, unsigned int height)` - Створює порожню текстуру заданого розміру. (Слайди 41-43)

— Демонстрація студентам використання графічної бібліотеки під час кодування в середовищі Microsoft Visual Studio. Повне коментування програмного коду з роз'ясненнями.

Виклад Практичного матеріалу матеріалу

- Завантажити презентаційний проект можна за посиланням:
https://drive.google.com/file/d/1eUUIhKqH4myHitAlx3TCAAXLEG_Q0g2/view?usp=sharing
- Приклад програмного коду з презентаційного проекту:

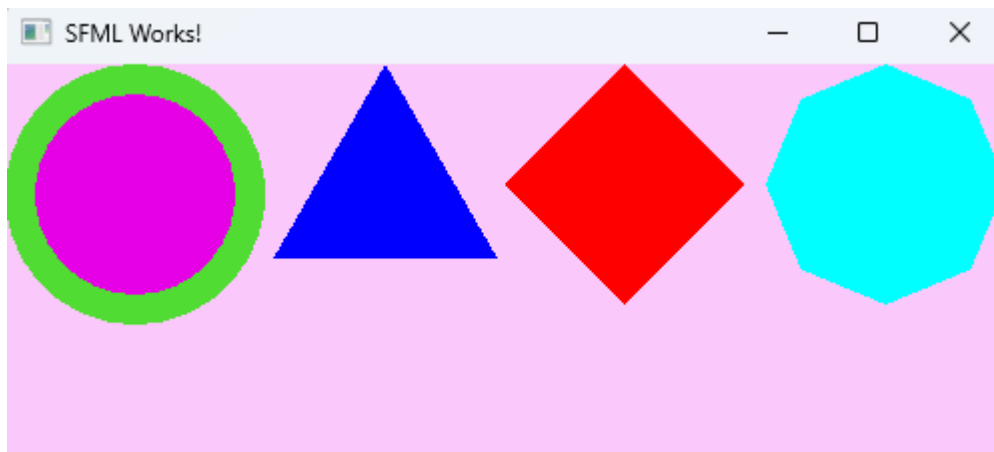
```
#include <SFML/Graphics.hpp>
using namespace sf; // підключаємо простір імен sf
int main()
{
    // створюємо вікно
    RenderWindow window(VideoMode(500, 200), "SFML Works!");
    // головний цикл, виконується поки відкрите вікно
    while (window.isOpen())
    {
        // опрацьовуємо чергу операцій в циклі
        Event event;
        while (window.pollEvent(event))
        {
            // якщо натиснули хрестик, закрити вікно?
            if (event.type == Event::Closed)
                window.close(); // тоді закриваємо
        }
        // Встановлення кольору фону
        window.clear(Color(250, 200, 250, 0));
        // Створюємо коло радіусом 50
        CircleShape circle(50.f);
        circle.setFillColor(Color(230, 0, 230)); // замальовуємо коло
```

```
circle.setOutlineThickness(15.f); // встановлення товщини кола
circle.setOutlineColor(Color(80, 220, 50)); // встановлення кольору контуру
circle.move(15, 15); // змінюємо позицію кола
window.draw(circle); // промальовуємо коло
// Створюємо трикутник
CircleShape triangle(65.f, 3);
triangle.setPosition(125, 0); // встановити позицію з початку
triangle.setFillColor(Color::Blue); // встановлення кольору
window.draw(triangle); // промальовуємо
// Створити квадрат
CircleShape square(60.f, 4);
square.setPosition(250, 0); // встановлюємо позицію від трикутника
square.setFillColor(Color::Red); // колір
window.draw(square); // промальовуємо

// створюємо октагон
CircleShape octagon(60.f, 8);
octagon.setPosition(380, 0); // встановлюємо позицію від квадрату
octagon.setFillColor(Color::Cyan); // колір "бірюза"
window.draw(octagon); // промальовуємо

// промальовуємо вікно
window.display();
}
return 0;
}
```

- Результат роботи програмного коду:



IV. Підведення підсумків. Рефлексія

- Сьогодні ми здійснили важливий перехід від суто консольного програмування до створення візуальних та інтерактивних додатків за допомогою графічної бібліотеки SFML. Головна мета заняття полягала в тому, щоб закласти міцний фундамент розуміння архітектури, яка керує кожною графічною програмою, а не лише запам'ятовуванні синтаксису.
- Ми з'ясували, що SFML (Simple and Fast Multimedia Library) є набором незалежних модулів (таких як System, Window, Graphics, Audio та Network), що забезпечує гнучкість і високу продуктивність, оскільки ми підключаємо лише необхідні нам функції. Центальною частиною нашої роботи став клас `sf::RenderWindow`, який є не просто рамкою на екрані, а цілцю рендерингу та нашим основним інтерфейсом взаємодії з операційною системою. Його правильна ініціалізація за допомогою `sf::VideoMode` та встановлення стилів (`sf::Style`) є першим кроком до створення будь-якої графічної сцени.
- Розуміння цих основ — циклу, обробки подій та розділення текстур і спрайтів — дає нам усі інструменти, необхідні для початку розробки власних 2D-додатків. Наступні заняття будуть присвячені ускладненню логіки, додаванню анімації та інтеграції інших модулів, але пам'ятайте: усе будується на цих фундаментальних принципах.

Обговорення відповідей та підсумок лекції

- Дякую вам за увагу, і ще раз хочу наголосити - пам'ятайте, що сьогодні ми побачили лише верхівку айсберга. Ви тепер знаєте, як створити сцену та розмістити на ній об'єкти, але ваша програма все ще статична.

V. Домашнє завдання

- Переглянути запропонованого під час лекції теоретичного матеріалу.
- Преглянути практичний проект.

Використані джерела:

1. López, A. Game Development with C++ and SFML. Birmingham: Packt Publishing, 2020. 300 с.
2. Hajny, J. Game Programming in C++: Creating 3D Games with SDL and OpenGL. Shelter Island: Manning Publications, 2020. 600 с.
3. Saks, A. The C++ Standard Library: A Tutorial and Reference. 3rd ed. Boston: Addison-Wesley, 2022. 1200 с.
4. Волков, В. В., Волкова, С. В. Комп'ютерна графіка: практикум. Харків: ХНУМГ ім. О. М. Бекетова, 2020. 154 с.
5. Нікітчук, Т. М., Стецюк, В. І. Паралельні обчислення та багатопотокове програмування. Вінниця: ВНТУ, 2022. 215 с.

Спеціалізоване програмне забезпечення

1. <https://visualstudio.microsoft.com/>
2. <https://www.sfml-dev.org/>